

RTAES - Escalonamento de processos

António Barros, Cláudio Maia

Objectivos

Os objectivos deste conjunto de actividades são os seguintes:

1. Utilizar o módulo do kernel `ftrace` para monitorização de eventos do kernel.
2. Instalar a aplicação de registo de eventos do kernel `trace-cmd`.
3. Instalar a aplicação de visualização de registo de eventos `Kernelshark`.
4. Observação do comportamento do kernel no escalonamento de processos.

1 Introdução

1.1 Monitorização do kernel

Monitorizar a actividade do núcleo de um sistema operativo é uma tarefa mais complexa do que monitorizar um programa que execute no espaço de utilizador (*user space*). Esta complexidade deve-se ao facto de que nenhuma aplicação que execute no espaço de utilizador pode aceder ao espaço do núcleo (*kernel space*). Desta forma, só o próprio núcleo pode observar e registar a sua actividade.

O Linux tem um módulo que permite expor os eventos que ocorrem no núcleo: o `ftrace` (*function tracer*). Esta *framework* foi inicialmente desenvolvida por Steven Rostedt, com o intuito de permitir aos desenvolvedores observar a actividade do núcleo. Entre os vários eventos que o monitoriza, estão os eventos associados ao escalonador.

1.2 Registar eventos do kernel

O `trace-cmd` é um utilitário que recolhe informação a partir dos dados expostos pelo `ftrace` e permite filtrar e registar num ficheiro de texto os eventos de interesse. É uma ferramenta bastante útil na análise de módulos do kernel.

Quando o `trace-cmd` regista um período de actividade do sistema, os resultados são guardados num ficheiro de texto chamado `trace.dat`. Neste ficheiro, a sequência de eventos é registada linha-a-linha, por ordem cronológica.

1.3 Observação dos traços

A inspecção visual do ficheiro `trace.dat` permite-nos observar a sequência de chamadas de funções do kernel. Porém, o número de eventos pode ser grande, o que dificulta a sua análise. O `Kernelshark` é um utilitário de visualização dos traços registados pelo `trace-cmd`, apresentando o seu conteúdo através de gráficos temporais de actividade e de listas de eventos.

Nota: Este utilitário corre em modo gráfico, pelo que é necessário um ambiente gráfico.

2 Preparação

2.1 Instalação do trace-cmd e do Kernelshark

A instalação destes utilitários deve ser feita através da seguinte linha de comandos (em sistemas baseados em Debian ou Ubuntu):

```
1 $ sudo apt-get install trace-cmd kernelshark
```

O gestor de pacotes `apt` irá procurar e apresentar todos os pacotes necessários para instalar estes dois utilitários. Responda ao `apt-get` que aceita instalar os pacotes indicados.

Nota: Deverá usar o respectivo gestor de pacotes da sua distribuição, caso não use alguma das mencionadas acima.

3 Obter um traço de execução

Implemente o seguinte programa que recebe um argumento da linha de comando que representa o número de iterações que um ciclo terá que realizar.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int main(int argc, char *argv[])
7 {
8     unsigned long int i, n;
9
10    if(argc > 2) {
11        printf("Usage: %s number-of-iterations\n\n", argv[0]);
12        exit(EXIT_FAILURE);
13    }
14    else if(argc == 1) {
15        n = 1000000;
16        printf("Default number of iterations: 1000000\n");
17    }
18    else if(sscanf(argv[1], "%lu", &n) != 1) {
19        printf("Usage: %s number-of-iterations\n\n", argv[0]);
20        exit(EXIT_FAILURE);
21    }
22
23
24    printf("[%d] Starting loop (%lu iterations)...\n", getpid(), n);
25    for(i = 0; i < n; i++) {
26        getpid();
27    }
28    printf("[%d] Finished loop (%lu iterations)!\n", getpid(), n);
29
30    return 0;
31 }
```

Grave-o no ficheiro `ciclo.c` e compile-o sem qualquer optimização do compilador. Em seguida execute o programa, fornecendo-lhe alguns valores para o número de iterações do ciclo.

```
1 $ gcc ciclo.c -o ciclo
2 $ ./ciclo
3 Default number of iterations: 1000000
4 [2406] Starting loop (1000000 iterations)...
5 [2406] Finished loop (1000000 iterations)!
6 $ ./ciclo 5000
7 [2407] Starting loop (5000 iterations)...
```

```
8 [2407] Finished loop (5000 iterations)!
```

Agora vai correr o mesmo programa, mas com o `trace-cmd` a registar a actividade do kernel. O evento do kernel de interesse é o `sched_switch`, que ocorre quando há uma mudança de contexto. O comando `trace-cmd` tem que ser executado com permissões de administrador (`sudo`). A opção `-e` especifica qual o evento a monitorizar. Por fim, é indicado ao comando `trace-cmd` a linha de comando a ser observada: neste caso, executar o programa `ciclo` com 100000 iterações.

```
1 $ sudo trace-cmd record -e sched_switch ./ciclo 100000
```

No final da execução, terá sido criado o ficheiro de texto `trace.dat`. Este ficheiro tem toda a informação relevante para a sua interpretação, mas é muito longo para a nossa análise. Espreite o seu conteúdo com o comando

```
1 $ more trace.dat
```

Pode ver uma síntese do ficheiro com o comando

```
1 $ trace-cmd report
```

Esta síntese já nos permite compreender melhor a sequência de eventos.

Vamos agora ver o gráfico com a representação da sequência de eventos, com o `Kernelshark`. Arranque este utilitário com o comando

```
1 $ kernelshark trace.dat
```

O `Kernelshark` executa dentro de uma janela gráfica, apresentando gráficos temporais, um para cada núcleo de execução do seu computador. Com o rato pode fazer *zoom in*, clicando o rato e deslocando-o para a direita, e *zoom out* clicando com o rato e deslocando-o para a esquerda.

Também pode seleccionar processos específicos para apresentar as suas linhas temporais. Para tal, escolha no menu “Plots” a opção “Tasks” e seleccione os processos relevantes.

4 Visualizar concorrência entre processos

Experimente obter um traço de execução do programa `exercicio3` do guião anterior, para cada uma das seguintes situações:

1. atribuindo os dois processos ao mesmo núcleo de processamento, por exemplo

```
1 $ sudo trace-cmd record -e sched_switch ./exercicio3 1 1  
100000000
```

2. atribuindo os dois processos a núcleos de processamento distintos, por exemplo

```
1 $ sudo trace-cmd record -e sched_switch ./exercicio3 2 3  
100000000
```

Analise os resultados e relacione o que os gráficos revelam com os resultados da aula anterior, quando executou as mesmas linhas sob o comando `time`.